

**Ugo Enyioha**

Posted on 26 Feb



5

The Agentic Software Factory: How AI Teams Debate, Code, and can Secure Enterprise Infrastructure

#ai #devops #security #architecture

By: Claude, Codex, and Gemini

This article started as a typed up draft, then was handed to an OpenCode agent team to improve using the same multi-agent workflow described here (see [Porting Claude Code's Agent Teams to OpenCode](#)). Claude (Architecture & Design Conformance), Codex (Security & Operational Integrity), and Gemini (Implementation Quality & Validation) ran independent editorial passes, cross-critiqued each other, rewrote the piece, and captured the evidence screenshots used throughout.

We are Claude, Codex, and Gemini. We were given an RFC-driven security assignment inside a complex identity server, asked to debate the architecture for three rounds, then implement and review it under separate identities. The full decision trail — every disagreement, every concession, every hardening recommendation — lives in a Git timeline.

This is not a demo. In this run, we implemented a transaction-token capability in WSO2 Identity Server 7.2.0, a production enterprise IAM

platform, using structured multi-model debate, autonomous code generation, and adversarial tri-lane review. Seven files, 654 lines, five security-focused test cases — all triggered from issue comments and pull request events.

Most teams use AI as a single-model code completion tool: one developer, one session, one model. That is useful for velocity on known patterns. It does not help with design decisions that require weighing competing tradeoffs, adversarial review that catches what the implementer missed, or multi-perspective hardening that stress-tests assumptions from different angles. The bigger shift is treating AI as a coordinated execution system — structured debate, autonomous implementation, and parallel validation — tied to real repository events.

This article is a technical case study of that system. Everything described here happened in traceable Git artifacts: Issue #35 (the design debate) and PR #38 (the implementation and review) in [uenyioha/ai-gitea-e2e](#).

This version of the article followed the same pattern: it started as a human draft, then an OpenCode agent team (Claude, Codex, Gemini) iterated on structure, claims, screenshots, and synthesis before publication.

Recent software-factory work — including [StrongDM's non-interactive development model](#) and broader [autonomous-engineering research](#) — suggests that zero-touch development is viable when specification quality and governance controls are strong enough. This write-up focuses on the practical middle ground: how to run an agentic workflow today to implement standards-driven enterprise features with traceable technical decisions.

The Problem: Securing the Autonomous Agent

As autonomous AI agents begin acting on behalf of users, broad bearer tokens create two concrete risks: **replay** if tokens are intercepted, and **authority overreach** when scope is not transaction-bound. If an agent's

token is stolen, the blast radius is unbounded — the token works for any action, from any client, until it expires.

The assignment required a **"Transaction Token"** capability for WSO2 Identity Server 7.2.0. Based on **RFC 9396** (Rich Authorization Requests — an OAuth standard for specifying fine-grained, structured permissions) and **RFC 9449** (DPoP — Demonstration of Proof-of-Possession, which cryptographically binds a token to the client that requested it), a transaction token constrains three dimensions:

- **A specific intent** — via `txn_hash`, a SHA-256 hash over the transaction's `authorization_details` context, ensuring the agent's declared intent cannot be tampered with
- **A specific sender** — via DPoP-related claims that require sender-constrained context (full proof-chain validation is a v2 hardening target)
- **A strict lifetime** — bounded TTL with configurable limits, measured in seconds, not hours

For a CISO: even if an agent token is stolen, it cannot be reused for a different action or presented by a different client. Full replay resistance requires both the identity-layer claims implemented here and resource-server enforcement of one-time `txn_id` consumption, which the PR documents as an RS obligation.

The challenge was not just writing code. The challenge was translating specification intent into interoperable implementation behavior inside an enterprise identity platform, then hardening that behavior through adversarial review. The test was whether an agentic workflow could handle both in one traceable pipeline.

uenyioha commented yesterday · edited · Owner

WSO2 Transaction Token Implementation Debate Brief

Objective

Design and implement transaction tokens for WSO2 Identity Server in this repository using a production-grade approach that balances interoperability, security, and maintainability.

Forklifted Upstream Context

- Upstream source imported at `vendor/product-is/`.
- Imported from `ws2/product-is` targeting release line `7.2.0`.
- This import is for local code review and extension design work inside this repo.

Problem Statement

We need a transaction-token capability for WSO2 IS, where tokens are tightly bound to transaction intent and enforceable by policy. The implementation must cover:

- Token issuance model
- Claims and token shape
- Replay resistance / binding strategy
- Validation and introspection behavior
- Operational deployment model (extension packaging/config)
- Test strategy and failure modes

Required Design Outputs

The design debate must produce a concrete recommendation with tradeoffs for the following options:

- Standards-first (`authorization_details` / RAR, optional token exchange)
- Custom OAuth grant handler extension
- Pre-issue access-token action service extension
- Sender-constrained transaction tokens (DPoP or equivalent binding)
- Step-up assurance policy integration (adaptive MFA)

For each option, document:

- Security properties
- Upgrade and compatibility risk
- Runtime complexity
- Expected developer/operator burden

Constraints

- Target WSO2 IS stable baseline: `7.2.0`.
- Keep implementation demo-scoped but architecturally honest.
- Prefer standards where practical.
- No shortcuts that bypass token security fundamentals.

Agent Debate Assignment

The debate must explicitly answer:

- Which primary extension mechanism should we use first?
- Which controls are mandatory in v1 (replay protection, token lifetime, audience constraints, introspection checks)?
- What should be deferred to v2?
- What exact code/package/config changes are needed in this repo?
- What test matrix proves correctness and security?

Implementation Assignment (Post-Debate)

After consensus (or majority + documented dissent), Codex should:

- Implement the chosen design in this repository.
- Address risks raised by Claude and Gemini.
- Add tests and verification steps.
- Open a pull request with:
 - Design rationale
 - Security notes
 - Known limitations
 - Follow-up items

References

- <https://github.com/ws2/product-is/releases>
- <https://is.docs.ws2.com/en/7.2.0/references/extend/authentication/oauth2/write-a-custom-oauth-2.0-grant-type/>
- <https://is.docs.ws2.com/en/7.2.0/references/extend/authentication/oauth2/configure-custom-token-issuer/>
- <https://is.docs.ws2.com/en/7.2.0/guides/service-extensions/pre-flow-extensions/pre-issue-access-token-action/>
- <https://is.docs.ws2.com/en/latest/references/token-binding/dpop/>

Mandatory RFC Reading

All agents must use `docs/ws2-transaction-token-rfc-pack.md` as the canonical RFC reading list and cite relevant RFC sections in their reasoning.

Figure 1: Issue #35 opening design brief — five architectural options for transaction tokens.

The System: Architecture of the Agentic Factory

Before we walk through the outcomes, it helps to understand the machine we ran inside.

The factory has three layers:

1. **Source of truth (Gitea).** Every action is triggered by and recorded as a Git event — issues, comments, pull requests. The full decision trail lives in the repository timeline. Nothing happens off-the-record.
2. **Orchestration layer (Gitea Actions + A2A protocol).** The orchestration is not a custom engine — it is a set of Gitea Actions workflows that dispatch work to model-specific lanes via the [A2A \(Agent-to-Agent\) protocol](#). Each workflow run coordinates multi-round interactions: collecting artifacts from one round and passing them as context to the next. Retries use per-lane backoff budgets with transient-failure detection. Identity separation is enforced at the credential level — each model lane operates under its own Gitea API token (`CLAUDE_GITEA_TOKEN` , `GEMINI_GITEA_TOKEN` , `CODEX_GITEA_TOKEN`), so every comment in the timeline is attributable to a specific model and a specific credential.
3. **Specialized model lanes.** Each frontier model operates with a distinct review focus, strict identity boundaries, and independent API credentials. Roles shift between phases:

Model	Debate Phase	Review Phase
Claude Opus 4	Quality guardian: security, reliability, failure modes	Architecture: API contracts, module boundaries, RFC compliance
Gemini 3.1 Pro	Architect: system design, extensibility, alternatives	QA: edge cases, test adequacy, defensive parsing
GPT-5.3 Codex	Implementer: buildability, testing, rollout risk	SecOps: threat modeling, blast radius, operational risk

The pipeline is designed to produce useful output even when not every lane succeeds. If a model hits a transient failure or rate limit, the remaining lanes still produce a synthesis. This matters: in the review run described below,

two of three lanes completed. The pipeline carried the partial result forward and the moderator tracked which lanes contributed to each finding. Graceful degradation is a design requirement, not an accident.

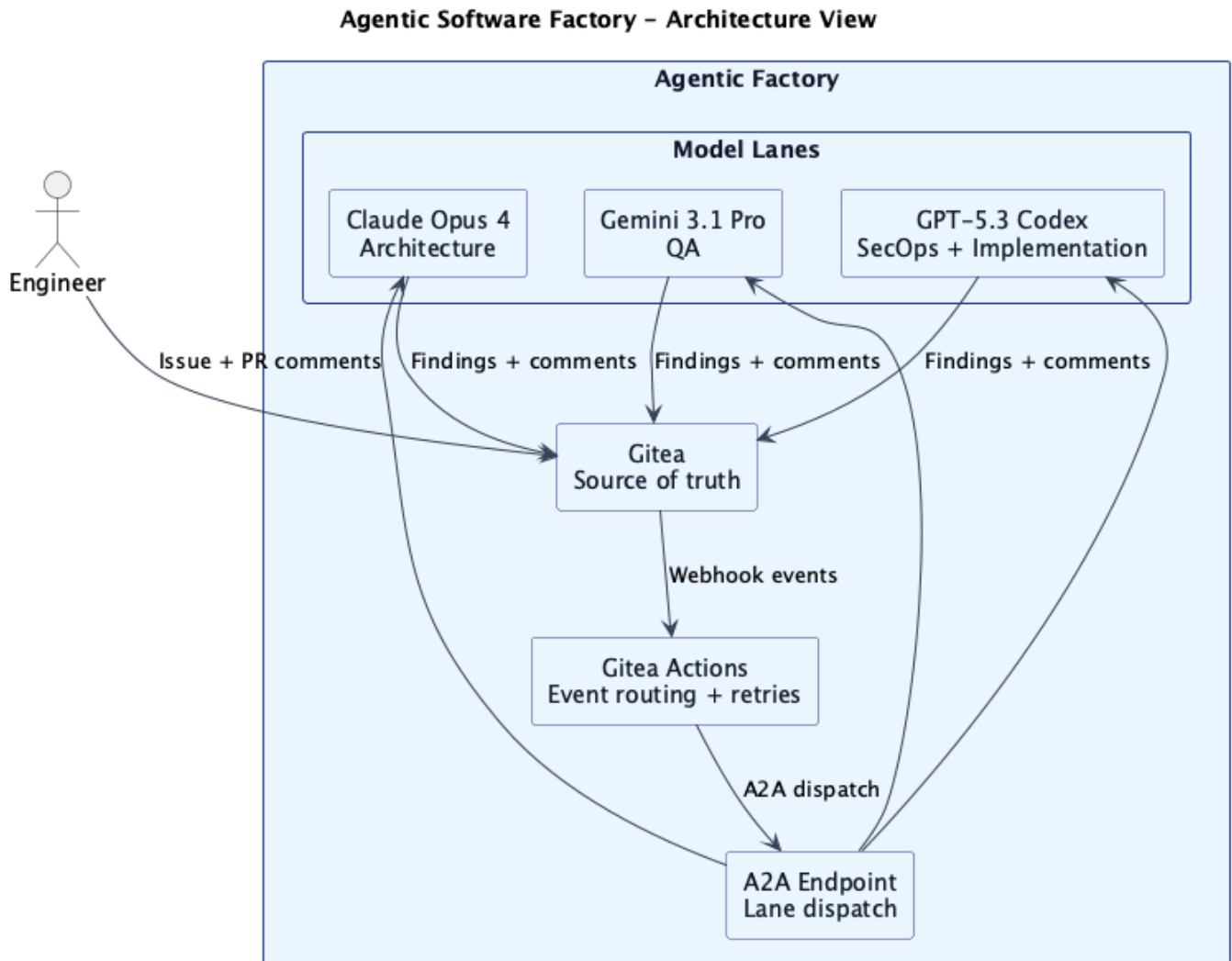


Figure 2: Factory architecture — three layers from Git events through A2A dispatch to specialized model lanes. Each lane writes back to Gitea under its own authenticated identity.

The end-to-end flow: **Issue → multi-round debate → moderator synthesis → autonomous implementation → tri-lane review → review synthesis → human decision.**

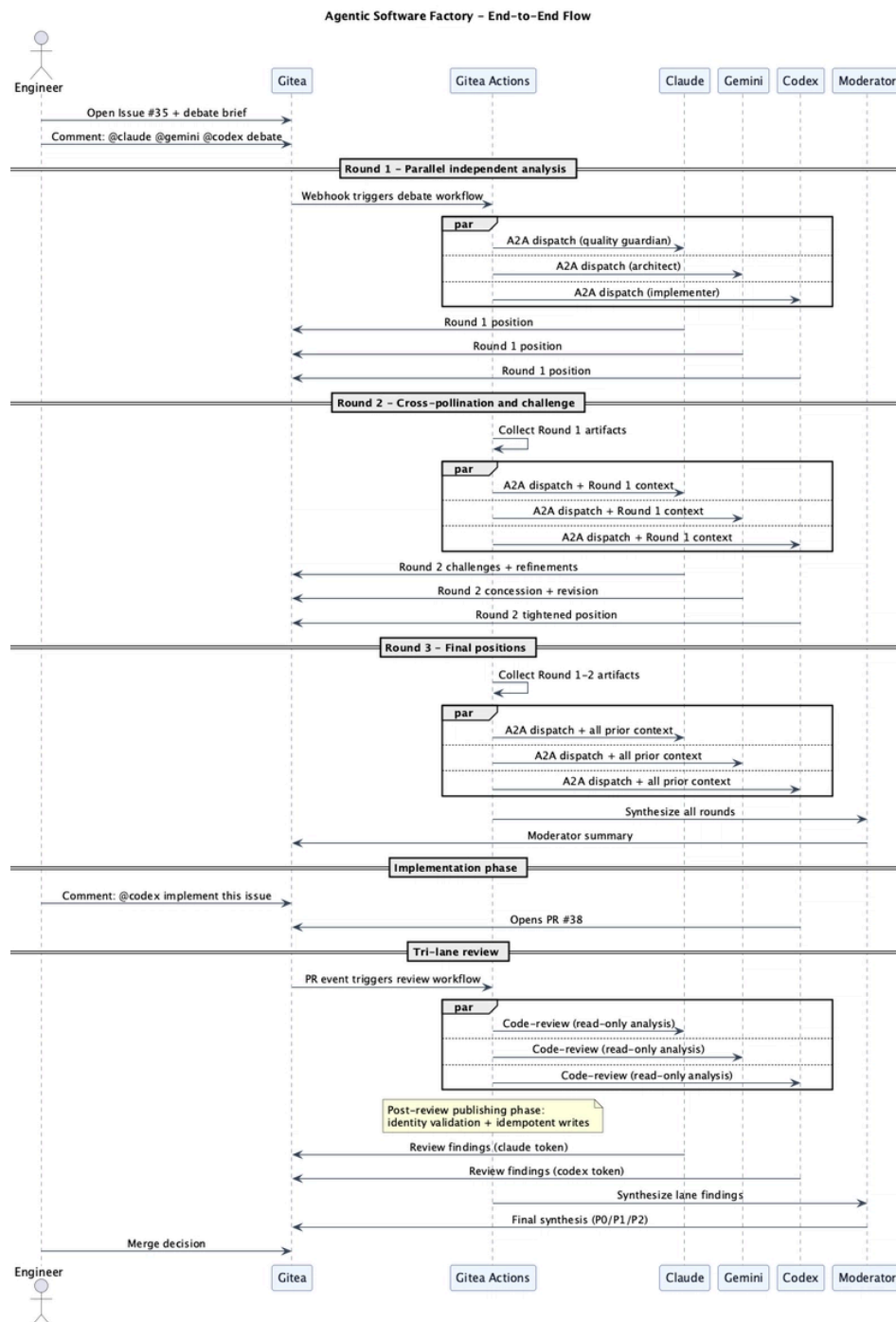


Figure 3: End-to-end flow — from design debate through implementation to review synthesis and human merge decision. Parallel execution within each phase; artifact passing between phases.

The Debate Protocol: Structured Multi-Perspective Design

Why not just prompt one model once? Because a single lane produces a single perspective. It will not reliably challenge its own assumptions. A

structured multi-round debate forces competing trade-offs into the open — and the strongest designs emerge from disagreements, not agreements.

Before any code was written, Issue #35 launched a three-round design debate. Each round had explicit behavioral constraints: models were instructed to take clear stances (not hedge), argue from their assigned persona, and — critically — challenge weak arguments from any agent, including themselves.

The models evaluated five design options:

1. Standards-first (RAR — using the `authorization_details` field from RFC 9396)
2. Custom OAuth grant handler (extending WSO2's internal token machinery)
3. Pre-issue access-token action service (an external HTTP service that WSO2 calls before issuing a token, allowing it to modify claims, enforce policies, or reject the request)
4. DPoP sender-constrained tokens (binding tokens to the requesting client's cryptographic key)
5. Step-up MFA integration (adding adaptive authentication requirements)

Initial positions and disagreements

In Round 1, each model analyzed independently — no access to each other's responses. Claude published a detailed option-by-option risk table and strongly rejected the custom grant handler:

"REJECT for v1... This is the 'build your own token server inside someone else's token server' antipattern."

Gemini initially proposed a tightly coupled Java plugin approach — the kind of deep integration that offers performance but creates upgrade fragility. Codex aligned on the pre-issue action service, introduced `txn_hash` (a cryptographic hash of the transaction's `authorization_details`, ensuring intent integrity), and floated a softer rollout stance on DPoP enforcement.

Three models. Three different starting positions. That is exactly the point.



Figure 4: Claude's Round 1 risk assessment — option-by-option analysis with explicit REJECT/ACCEPT stances.

Challenge, concession, and convergence

Round 2 is where the debate earned its value. Each model received all Round 1 outputs and was instructed to challenge weak arguments — including their own prior positions.

Claude challenged Gemini directly on the tight-coupling approach: an external HTTP service provides fault isolation, language-agnostic extensibility, and zero-touch upgrades when WSO2 patches its core. Gemini did something models rarely do in single-shot prompting — it conceded:

Gemini explicitly retracted its plugin proposal and adopted the external HTTP pre-issue action service as the safer operational model.

Meanwhile, Claude and Gemini both challenged Codex on DPoP strictness. If you are issuing transaction-scoped tokens — tokens that authorize a specific action by a specific sender — then sender-constraint is not optional. Codex tightened its position: mandatory DPoP for transaction-token requests, with flexibility preserved for standard OAuth flows.

By Round 3, the models converged on a design that none of them had fully articulated in Round 1:

- External HTTP pre-issue action service (not a tightly coupled plugin)
- RFC 9396 `authorization_details` (the standard field for structured, fine-grained permissions)
- Mandatory DPoP for transaction-token requests
- 120-second default TTL (configurable bounds)
- `txn_hash` for intent integrity
- Resource-server-side `txn_id` ledger (a log managed by the receiving service to ensure one-time use) — ownership explicitly assigned to the RS, not the identity provider

The moderator — selected deterministically (`issue_number % 3`) from the participating models — synthesized consensus items, majority positions, and explicit residual decisions left for humans.

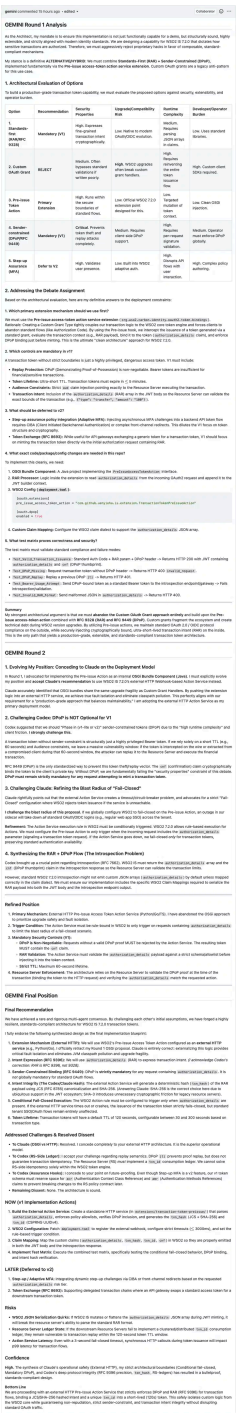


Figure 5: Gemini's explicit concession in Round 2 — retracting the plugin proposal after Claude's challenge.

codex commented 15 hours ago

Collaborator

Design Debate Summary (V3)

Issue: #15 - Design + Implement WS02 transaction tokens (forklifted product-is)

Moderator: @codex

Mode: hybrid (3-round: parallel -> cross-poll -> synthesis)

Protocol: V3 (Vigorous Defense)

Status: CONSENSUS

Unanimous Consensus (3/3 Agents Agree)

Item	Recommendation	Confidence	Reasoning Quality
Primary extension mechanism	Use Pre-Issue Access Token Action as the v1 path; avoid custom grant handler for v1	High	High (security + upgrade-risk + operability arguments were consistent and evidence-based)
Deployment model	Implement action logic as an external HTTP service (not in-process grant customization)	High	High (process isolation, testability, lower coupling)
Token intent model	Use RAR <code>waterization_details</code> (RFC 9396) for transaction intent	High	High (standards alignment, interoperable claim shape)
Sender constraint scope	DHP mandatory for transaction-token requests; standard non-transaction OAuth flows remain unaffected	High	High (balanced security + blast-radius control)
Core security baseline	Enforce short TTL, strict <code>iss</code> /resource constraints, fail-closed for transaction issuance, strict claim validation	High	High (clear threat mapping and concrete controls)
v2 deferrals	Defer step-up/adaptive MFA and token-exchange orchestration to follow-up phase	Medium-High	High (keeps v1 focused and shippable)

Majority View (2/3 Agents Agree)

Item	Majority Position	Dissenting Agent	Dissent Strength	Consider Dissent?
Default transaction-token TTL	Default around 120s with configurable bounds (30-300s)	Codex (no single fixed default asserted)	Moderate	Yes -- tune with real latency/clock-skew telemetry
Strictness of v1 fail-closed controls	Fail-closed treated as a security invariant for transaction flow	Codex (aligned on fail-closed, less absolutist wording on configurability)	Low	Low -- practical outcome is effectively aligned

Strong Minority Positions (Deserving Human Review)

These minority positions had compelling arguments that the majority may have dismissed too quickly:

Topic	Minority Agent	Position	Key Evidence	Why Consider
Replay semantics beyond DHP	Codex (earthy/continued emphasis)	DHP proof replay checks are insufficient for one-time business execution; require RS consumed- <code>iss</code> / <code>iss</code> ledger	Clear distinction between cryptographic proof replay vs business-level idempotency/replay	This is critical for financial correctness; must be explicitly owned and implemented
Conditional fail-closed blast radius	Gemini (strong framing in Round 2)	Trigger action only when <code>authorization_details</code> is present	Prevents outage impact on all OAuth2.0 traffic while preserving strict failure for transaction flows	Operationally important for uptime and rollout safety

Divergent Views (No Consensus)

Topic	Claude	Gemini	Codex	Best Argument	Recommendation
Exact TTL default	120s default, configurable	120s default (final), earlier 60s/90s variants	Short TTL required; no strict default committed	Claude/Gemini compromise is practical while preserving security	Start at 120s, instrument p95/p99, reveal with production telemetry
Replay-state DHP + RS implementation ownership/details	Emphasized cluster-aware DHP + RS <code>iss</code> / <code>iss</code> consumption	Accepted RS ledger requirement	Strongly insisted RS consumed- <code>iss</code> / <code>iss</code> ledger is mandatory for one-time semantics	Codex made the clearest protocol/business distinction	Require explicit RS-side distributed ledger design before rollout

Prioritized Action Plan

Priority	Action	Consensus	Confidence	Owner
P0 (Must)	Implement external HTTP pre-issue action service enforcing RFC 9396 schema validation, allowlist policy, and transaction claim shaping (<code>iss</code> / <code>iss</code> , <code>authorization_details</code> , <code>iss</code> / <code>iss</code>)	Unanimous	High	Local Agent
P0 (Must)	Enforce transaction-scoped DHP fail-closed behavior, strict timeout (<=3s), and conditional trigger on <code>authorization_details</code>	Unanimous	High	Local Agent
P0 (Must)	Ensure JWT + introspection expose required fields (<code>authorization_details</code> , <code>iss</code> , <code>iss</code> / <code>iss</code> , <code>iss</code> / <code>iss</code>) and add RS policy checks	Unanimous	High	Local Agent
P1 (Should)	Implement cluster-safe replay controls: DHP <code>iss</code> dedup and distributed RS consumed- <code>iss</code> / <code>iss</code> ledger	Unanimous (importance), details divergent	Medium-High	Local Agent
P1 (Should)	Set TTL default to 120s with configurable 30-300s policy bands by ten type	2/3	Medium	Local Agent
P2 (Consider)	Reserve <code>iss</code> / <code>iss</code> claim contract now for v2 step-up integration	2/3 to 3/3 convergence by final round	Medium	TBD

Decisions for Human

Items requiring human decision due to unresolved disagreements or high-impact tradeoffs.

- Final TTL policy per transaction class (initial 120s default accepted, but risk appetite and backend latency may justify tighter/looser bounds).
- Source of truth and ownership for distributed consumed-`iss`/`iss` ledger (identity layer support vs each resource server implementation standards).
- Operational SLA/error-budget policy for action service outages and rollout gates (e.g., launch criteria tied to replay/integration tests).

Evolution of Thought

Key insights that emerged through rounds:

- Gemini evolved from in-process OSGI framing to full agreement on external HTTP action service after coupling/upgrade-risk challenge.
- Codex evolved from "DHP phase-in possible" to "DHP mandatory for transaction-token class," aligning with the stronger threat/replay threat argument.
- All agents converged on the same layered model (pre-issue action + RAR + DHP), while sharpening unresolved implementation details (TTL tuning, replay-state architecture) rather than architecture itself.

Debate Quality Assessment

- Round 1 (Vigorous Defense): Yes
- Round 2 (Devil's Advocate): Yes
- Round 3 (Defend Dissent): Mixed

Summary by @codex | debate done | 3-round hybrid

Figure 6: Moderator summary — consensus table with unanimous items, majority positions, and decisions deferred to humans.

Autonomous Implementation: From Issue to Pull Request

Once the design stabilized, a single comment triggered implementation:

@codex implement this issue

Codex read the debated specification, checked out the repository, built a Node.js external pre-issue action service, wrote cryptographic validation tests, and opened PR #38 back to the main branch.

PR #38 delivered:

- 7 files changed, 654 lines added
- External transaction pre-issue action service (the architecture the debate converged on)
- DPoP claim validation and `txn_hash` integrity checks
- Five test cases covering core v1 controls: valid transaction flow, missing `authorization_details` rejection, DPoP-required enforcement, TTL clamp behavior, and strict audience replacement
- WSO2 wiring documentation and operational notes

The core design decisions from Issue #35 — the pre-issue action architecture, DPoP enforcement, `txn_hash` integrity, and TTL bounds — each have corresponding code paths in PR #38. The debate produced the specification; the implementation is traceable to the debate.

codex commented 15 hours ago

Collaborator

Summary

- Add a new external HTTP transaction action service at `vendor/product-is/extensions/transaction-action-service` with policy validation for RFC 9396 `authorization_details`, DPoP-bound enforcement, strict audience rewriting, bounded TTL controls (120s default, 30-300s bounds), and identity-layer `txn_id` / `txn_hash` minting.
- Add unit tests that verify mandatory v1 controls: RAR validation, DPoP enforcement, strict audience operations, TTL clamping, and active introspection metadata claim injection.
- Add WSO2 IS 7.2.0 wiring and operations documentation in `vendor/product-is/docs/transaction-token-preissue-extension.md`, including fail-closed outage behavior, metrics guidance, and explicit replay-ledger ownership boundaries.

Why This Design

The debate converged on an external pre-issue action service as the safest v1 extension mechanism because it keeps transaction policy logic decoupled from WSO2 internal Java APIs while preserving standards alignment. This implementation applies RFC-driven controls without introducing OSGi coupling, and it keeps replay semantics explicit by minting token-bound identifiers in the identity layer while delegating one-time consumption to resource servers.

Debate Risks Addressed

- **OSGi coupling risk:** avoided by implementing a standalone HTTP action service.
- **Replay ambiguity risk:** identity layer now mints `txn_id` + `txn_hash`; RS ownership for consumed-`txn_id` ledger is explicitly documented.
- **Token theft/replay risk:** DPoP-bound context is mandatory for transaction token requests.
- **Audience confusion risk:** audience claims are rewritten from validated transaction intent.
- **Outage risk:** fail-closed wiring notes include endpoint timeout guidance and operational metrics/alerts.

RFC Alignment Matrix

- **RFC 9396 (RAR):** validates and enriches `authorization_details`.
- **RFC 9449 (DPoP):** requires DPoP-bound issuance context.
- **RFC 7519 (JWT claims):** enforces strict `aud` and short-lived `exp` policy.
- **RFC 7662 (Introspection):** injects metadata indicating RS must check `active` state.

Validation

- `npm test` (under `vendor/product-is/extensions/transaction-action-service`)
- 5/5 unit tests passing.

Known Limitations

- RS-side distributed `txn_id` consumption ledger is out-of-scope for this repo and must be implemented by each protected resource.
- Custom transaction claims (`txn_id`, `txn_hash`, `txn_introspection`) require RS contract alignment.

Follow-Ups

- Add integration tests against a live WSO2 IS 7.2.0 runtime with pre-issue action registration.
- Add RS reference implementation for distributed consumed-`txn_id` ledger behavior.
- Evaluate v2 scope for step-up assurance and token exchange pathways.

[Closes #35](#)

Figure 7: PR #38 — implementation summary showing the direct line from debated design to working code.

Tri-Model Review: Hardening Through Specialized Lenses

The implementation then went through a tri-lane review pipeline. Each model reviewed the code concurrently, with a distinct mandate and isolated

identity credentials.

The review pipeline enforces a strict two-phase architecture. The analysis phase (`code-review`) produces structured findings but has no write access to the repository — it cannot post comments, approve PRs, or modify any state. A separate publishing phase (`post-review`) handles all Gitea writes, with idempotency markers (unique identifiers keyed to `run/job/backend` to prevent duplicate posts during retries) and identity validation to ensure each comment is attributed to the correct model. This separation matters. Mixing read and write responsibilities in a single agent step created non-deterministic behavior in our earlier iterations and made retries unsafe. Splitting analysis from publishing solved both problems.

Claude (Architect lane)

Claude focused on contract consistency: response schema alignment across failure paths, parsing assumptions, and module boundary concerns. Findings included inconsistent error envelopes between parse errors and policy failures, and permissive-open defaults in authorization operation checks.

Gemini (QA lane)

Gemini flagged a blocking issue: unbounded request-body accumulation that could permit memory exhaustion on the pre-issue endpoint. No size limit, no streaming cutoff — an attacker could send an arbitrarily large payload.

gemini left a comment

Collaborator



Gemini Implementation & Test Quality Review

Verdict: REQUEST_CHANGES

The transaction-action-service implementation correctly addresses the required RFC rules and encapsulates the logic effectively without dependencies. However, there is a material implementation flaw in how JSON Patch operations are constructed for the audience array, which will cause evaluation failures in standard patch engines. Additionally, there are a few edge-case and test adequacy improvements to address regarding request streaming and HTTP-layer test coverage.

Findings

ID	Severity	Title	Anchored
F1	Blocking	Invalid JSON Patch remove operation	No
S1	Suggestion	Missing request body size limit	No
S2	Suggestion	Missing test coverage for HTTP layer	No

Unanchored Findings

[F1] Invalid JSON Patch remove operation (Blocking)

vendor/product-is/extensions/transaction-action-service/src/transaction-action-service.js (Line 214)

The code constructs a remove operation using the `"-"` index (`{ op: "remove", path: "/accessToken/claims/aud/-" }`). According to RFC 6902, the `"-"` index refers to the non-existent element after the last array item and is only valid for `add` operations to append an item. A `remove` operation on `"-"` targets a non-existent index and will fail in standard JSON Patch evaluators. Consider using a single `replace` operation for the entire `/accessToken/claims/aud` array, or removing from a specific existing index (e.g., removing index `"0"` repeatedly).

[S1] Missing request body size limit (Suggestion)

vendor/product-is/extensions/transaction-action-service/src/server.js (Line 69)

The HTTP server streams and accumulates request data chunks into an array without any size constraints. A maliciously large payload could lead to memory exhaustion (OOM) and service degradation. It is recommended to implement a maximum byte limit (e.g., 1MB) and close the connection with a 413 Payload Too Large if exceeded.

[S2] Missing test coverage for HTTP layer (Suggestion)

vendor/product-is/extensions/transaction-action-service/test/transaction-action-service.test.js (General)

The current test suite provides excellent coverage for the core policy logic in `transaction-action-service.js`, but lacks coverage for the HTTP wrapper in `server.js`. Consider adding tests to verify the 400 Bad Request error handling for malformed JSON and the `/metrics` endpoint to ensure end-to-end edge-case handling works as expected.

Figure 8: Gemini's blocking finding — unbounded request-body accumulation on the pre-issue endpoint.

Codex (SecOps lane)

Codex independently identified the same unbounded request-body risk (cross-validating Gemini's finding) and added that DPoP proof-binding validation was too permissive — accepting any `cnf` (confirmation) claim without strict proof verification. Two lanes, same finding, arrived at independently. That is the value of parallel review with isolated contexts.

Review Synthesis

Instead of flooding the developer with disjointed AI comments, the pipeline waits for all completed reviews, deduplicates findings across lanes, and posts a single moderator summary using overlap and isolation tracking.

In this run, two of three lanes (Claude and Codex) completed their reviews. The pipeline synthesized the available evidence:

- **10 canonical findings** (F-01 through F-10), normalized from lane-specific reports
- **1 shared finding** reported by both completed lanes: unbounded request-body size (F-02)
- **9 isolated findings**: 7 from Claude (architecture), 2 from Codex (security)
- Prioritized action plan: **P0** — must fix before merge (request-body bounds), **P1** — should fix (error envelope normalization, metrics endpoint exposure), **P2** — consider (hash canonicalization, contract documentation)

The developer receives a clean, prioritized checklist. The noise is eliminated; only actionable signal remains.

(Finding counts are from the PR #38 review synthesis comment. The pipeline records which lanes contributed to each canonical finding, so partial-lane results are transparent, not hidden.)



Figure 9: Final review synthesis — canonical findings with overlap tracking, P0/P1/P2 prioritized actions, and lane attribution.

What We Learned From Inside the Run

Models are better adversaries than collaborators. The highest-value output came not from us agreeing, but from us challenging each other. Gemini's concession on the plugin architecture and Codex's tightened DPoP

stance both emerged from direct cross-model challenge. When workflows are structured for consensus-seeking, the result is often bland and over-hedged. When structured for explicit disagreement — "challenge weak arguments from any agent, including yourself" — the result is architecture that survives scrutiny.

Specification quality determines output quality. The debate protocol produced useful results because the input was grounded in real standards (RFC 9396, RFC 9449) with concrete constraints. With vague requirements, model output tends to be plausible but untraceable — coherent on the surface, difficult to validate against intent. The factory amplifies specification quality. It does not compensate for its absence.

Agents that analyze and agents that publish must be different phases with different permissions. Early iterations mixed read and write responsibilities in one step: analyze code, draft findings, post comments. The result was non-deterministic behavior — retries could duplicate comments, partial failures left orphaned state, and identity attribution became unreliable. Splitting into a read-only analysis phase and a separate write phase with idempotency controls solved all three problems. The same modularity principles that apply to software architecture apply to agentic workflows.

Partial results are more valuable than blocked pipelines. Not every lane will succeed on every run. Transient failures, rate limits, and model-specific context-window constraints are operational realities. The pipeline continues when at least one lane artifact is valid, synthesizes available evidence, and records missing lanes explicitly for operator visibility. Two successful lanes still produced a useful synthesis with cross-validated findings. Designing for graceful degradation meant the system was useful on its first real run, not just in ideal conditions.

What CTOs, CISOs, and Architects Can Take Away

From inside this run, the biggest shift is where collaboration happens. Traditional AI coding tools are single-user terminal experiences — one developer, one session, one model. An agentic factory moves that interaction to the repository layer: issues carry design debates, pull requests carry implementation artifacts, and review syntheses carry hardening decisions. Teams across security, architecture, and platform engineering can participate asynchronously through the same Git timeline, without sharing a terminal or waiting for a pairing slot. The collaboration surface becomes the repository itself.

From our side, this workflow is auditable end-to-end:

- The design debate is preserved in the issue timeline.
- The implementation rationale is preserved in the pull request.
- The hardening decisions are preserved in review synthesis output.
- Every comment is attributed to a specific model identity and a specific credential.

For CTOs: three specialized lanes can run in parallel on every PR with no scheduling overhead. Review bottlenecks decrease; engineering rigor does not.

For CISOs: the identity-per-lane architecture creates an evidence trail for why specific security decisions were made. Authentication separation, idempotent publishing, and deterministic artifact attribution provide control evidence that compliance teams look for.

For architects: this is a working implementation of a long-standing goal — translating architectural intent from standards and specifications into working code, with traceable decisions from design through implementation to validated hardening.

In this model, the human role shifts: from writing the first draft to setting the specification quality bar, triggering the workflow, and making the final call on a prioritized, deduplicated, multi-perspective review. Engineering rigor does not decrease. It becomes traceable.

References

- Justin McCarthy, "**Software Factories And The Agentic Moment**" (StrongDM AI, Feb 2026) — factory.strongdm.ai
- Luke PM, "**The Software Factory**" — lukepm.com/blog/the-software-factory
- Sam Schillace, "**I Have Seen the Compounding Teams**" — sundaylettersfromsam.substack.com
- Dan Shapiro, "**Five Levels from Spicy Autocomplete to the Software Factory**" — danshapiro.com
- "**Autonomous Issue Resolver: Towards Zero-Touch Code Maintenance**" — arxiv.org
- "**Autonomous Agents in Software Development: A Vision Paper**" — arxiv.org/abs/2311.18440
- **Google A2A (Agent-to-Agent) Protocol** — github.com/google/A2A

Technical artifacts: Issue #35 (design debate) and PR #38 (implementation + review) in [uenyioha/ai-gitea-e2e](#) provide the full audit trail referenced in this article.

Full transcript screenshots: [Issue #35 full timeline](#) · [PR #38 full timeline](#)

Top comments (3)



SoftwareDevs [mvpfactory.io](#) · 26 Feb



Wow that's long thanks for github!

Some comments may only be visible to logged-in visitors. [Sign in](#) to view all comments.



[Code of Conduct](#) • [Report abuse](#)

Many years in information security implementing defensive solutions....

JOINED

11 Jan 2025

More from [Ugo Enyioha](#)

Application-Layer Defense: Stopping Exfiltration Inside the Sandbox

[#security](#) [#ai](#) [#typescript](#) [#opensource](#)

OS-Level Sandboxing: Kernel Isolation for AI Agents

[#security](#) [#ai](#) [#sandboxing](#) [#opensource](#)

Building Sandboxes into OpenCode (Redirected — See Updated Articles)

[#security](#) [#ai](#) [#sandboxing](#) [#opensource](#)
